University of Nottingham
UK | CHINA | MALAYSIA

COMP2054 Tutorial Session 6: Heaps, BSTs, and BST Rotations

Warren Jackson
Ian Knight
Cas Widdershoven

Session outcomes

- Understand difference between heaps and BSTs.
- Know how to apply the various operations on heaps and BSTs.
- Be able to balance BSTs using rotations.

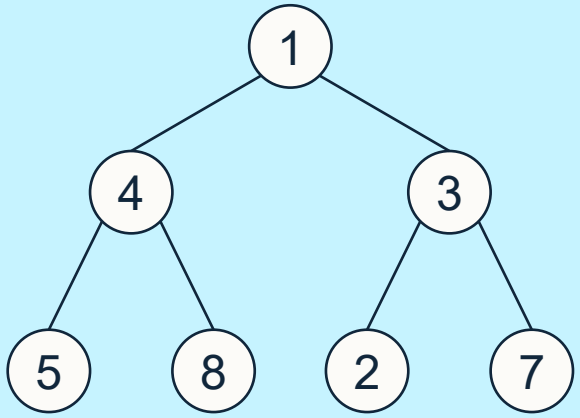
University of Nottingham
UK | CHINA | MALAYSIA

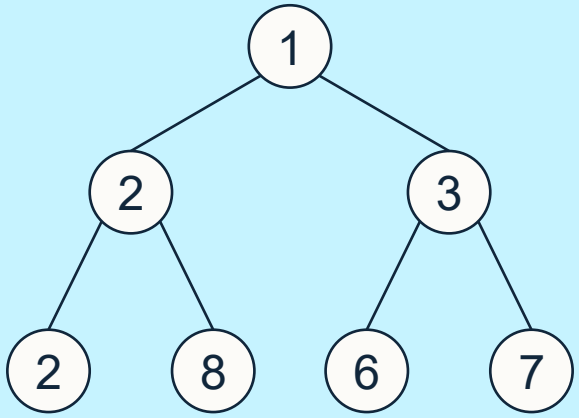
Trees

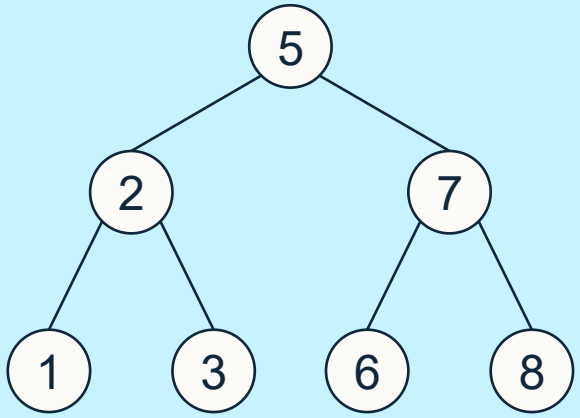
Heaps and BSTs

Quiz – Q1

Classify each of the following trees as a Heap, a BST, or neither:



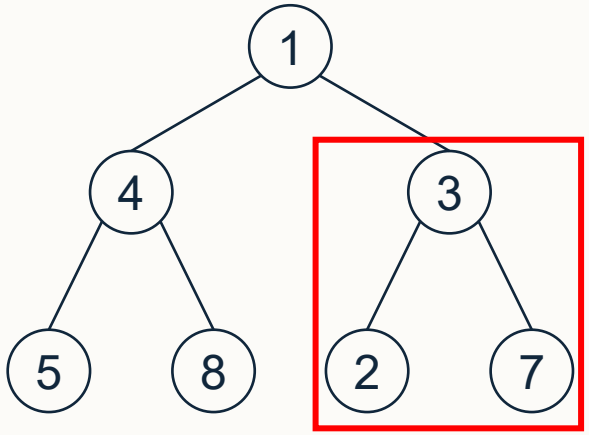




Quiz – Q1

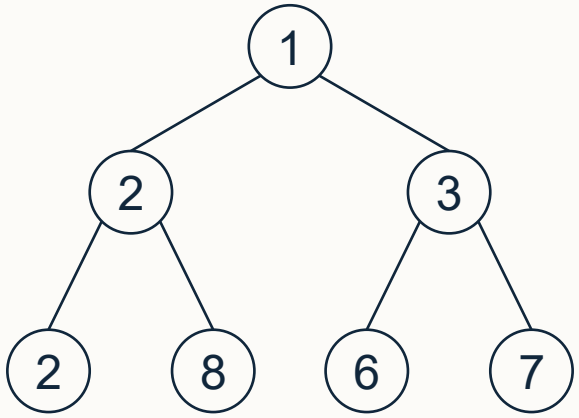
Classify each of the following trees as a Heap, a BST, or neither:

Neither



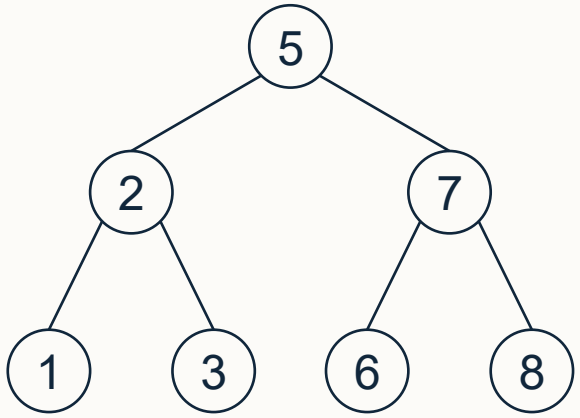
While at first this appears to be a heap with the smallest element (1) at the root, the highlighted subtree does not have the minimum element (2) at its root which is (3).

Heap



This is a valid heap. The tree has the minimum element (1) at its root, and all sub-trees contain the smallest element at their roots.

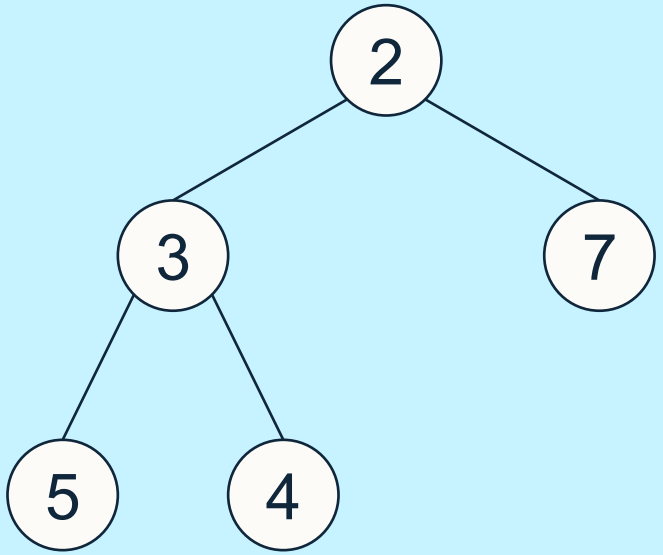
BST



This is a valid BST since for all nodes, their left subtree contains only values less than the given node, and the right subtree only values greater than the given node. E.g. max[-,2,1,3] < 5 < min[-,7,6,8]

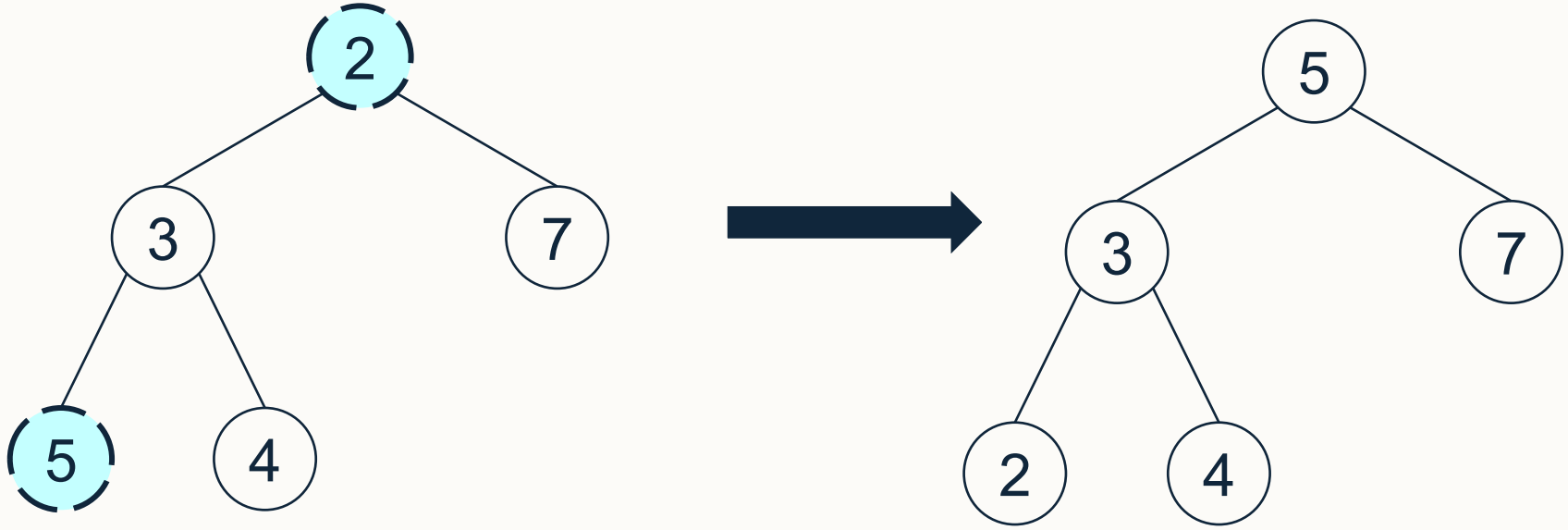
Quiz – Q2

Which two nodes in the following heap should be swapped to create a BST?



Quiz – Q2

Which two nodes in the following heap should be swapped to create a BST?

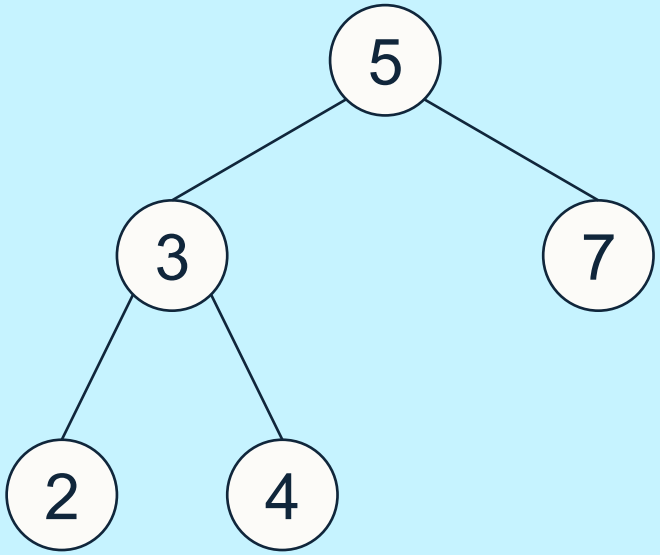


Swapping the 2 and the 5 will result in the binary tree [-,5,3,7,2,4] which is a BST.

Array Representation of Binary Trees

How can we represent the below BST in an array?

Index	[0]	[1]	[2]	[3]	[4]	[5]
Value						



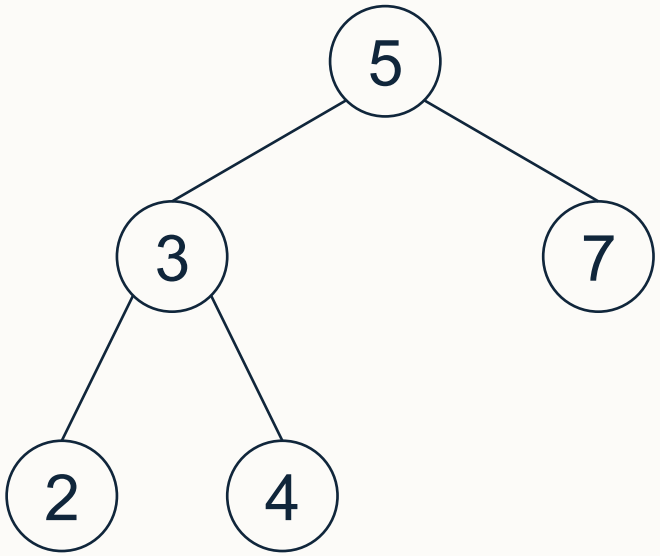
Remember:

```
index(root) = 1
index(left_child, i) = 2i
index(right_child, i) = 2i + 1
```

Array Representation of Binary Trees

How can we represent the below BST in an array?

Index	[0]	[1]	[2]	[3]	[4]	[5]
Value		5	3	7	2	4



Remember:

```
index(root) = 1
index(left_child, i) = 2i
index(right_child, i) = 2i + 1
```

Quiz – Q3

Given the two binary trees in an array-based format, identify which is a heap and which is a BST, and explain why.

Binary tree 1:

Index	`[0]`	`[1]`	`[2]`	`[3]`	`[4]`	`[5]`	`[6]`	`[7]`
Value		2	3	5	4	6		

Binary tree 2:

Index	`[0]`	`[1]`	`[2]`	`[3]`	`[4]`	`[5]`	`[6]`	`[7]`
Value		4	3	5	2			6

Quiz – Q3

Given the two binary trees in an array-based format, identify which is a heap and which is a BST, and explain why.

Binary tree 1:

Index	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
Value		2	3	5	4	6		

This cannot be a BST since the root contains the smallest element and has a left child (3) which is not less than or equal to itself. Drawing out the tree, we can see that this **is** a valid heap.

Binary tree 2:

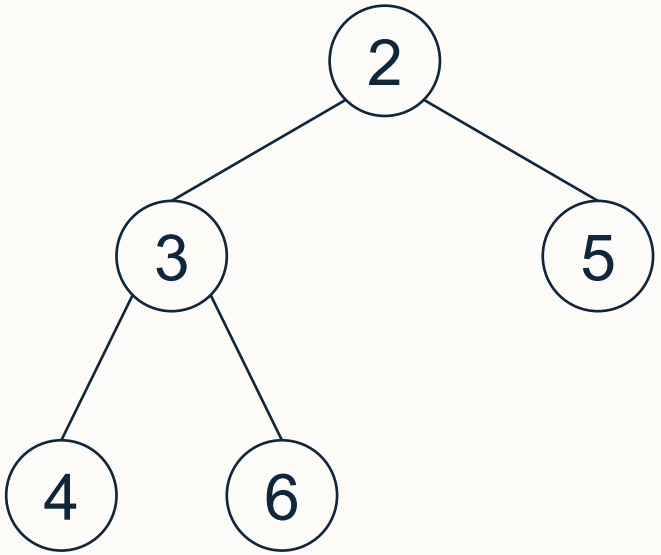
Index	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
Value		4	3	5	2			6

This cannot be a heap since the root does not contain the smallest element. Furthermore, there are “gaps” meaning the shape property of the heap is violated. Drawing out the tree, we can see this **is** a valid BST.

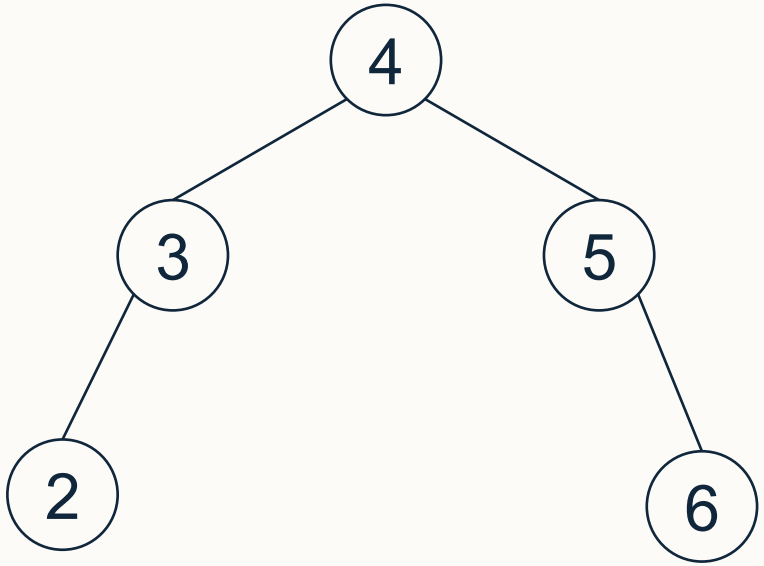
Quiz – Q3 (Binary Trees 1 & 2 as trees)

Given the two binary trees in an array-based format, identify which is a heap and which is a BST, and explain why.

Binary Tree 1

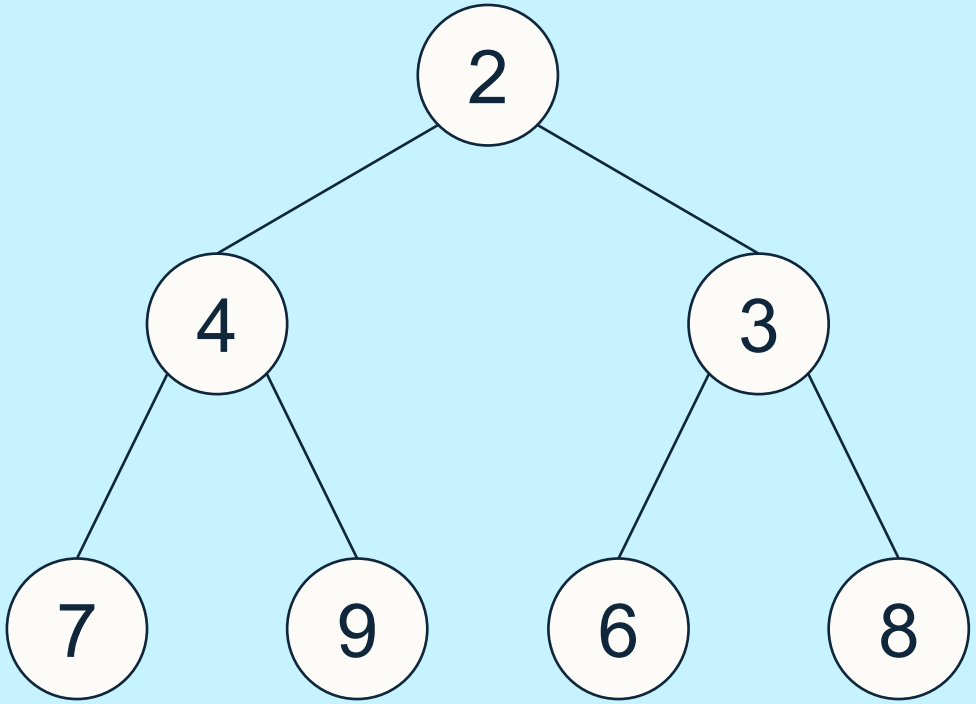


Binary Tree 2



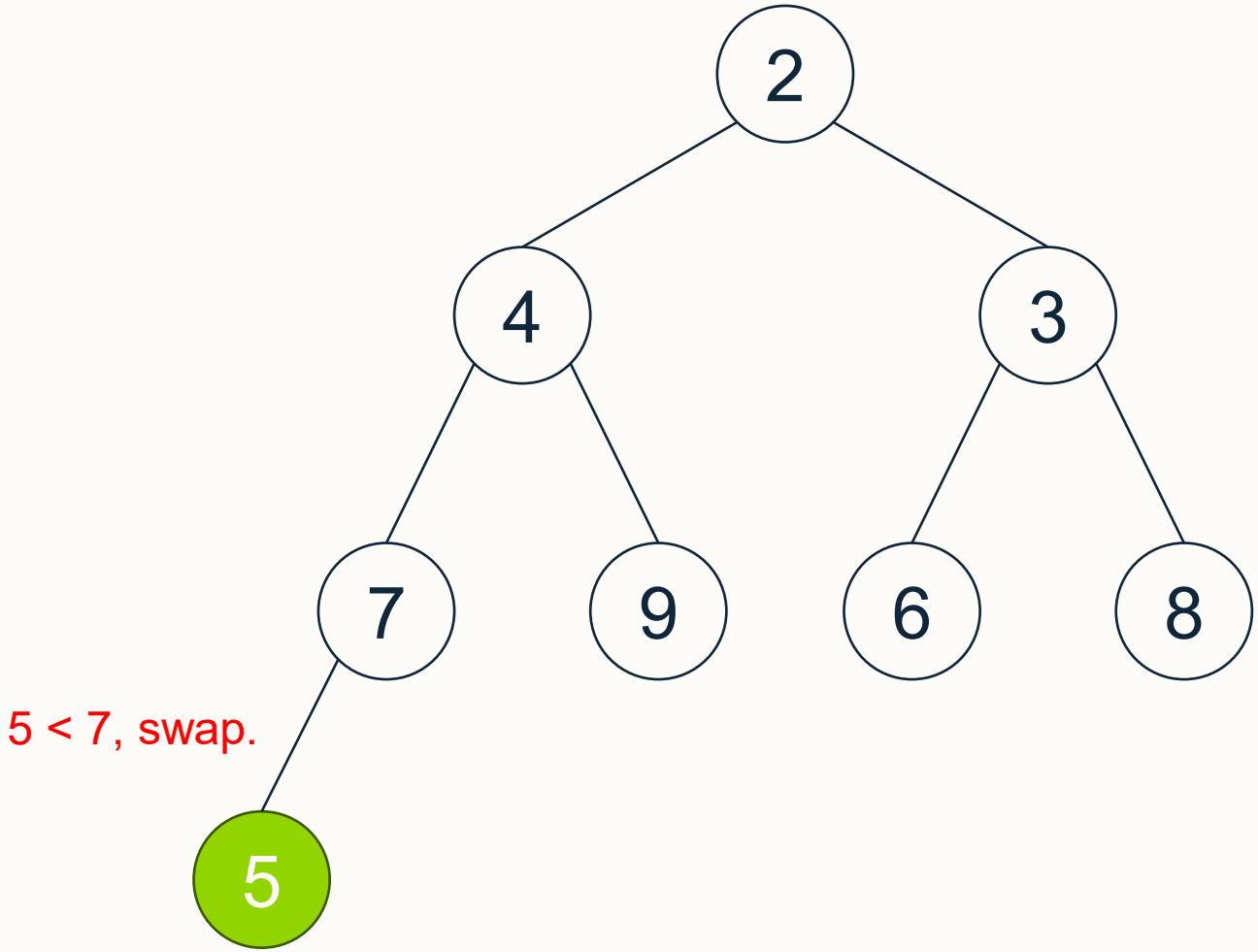
Operations on Heaps – insertItem(x)

- Starting with the heap:



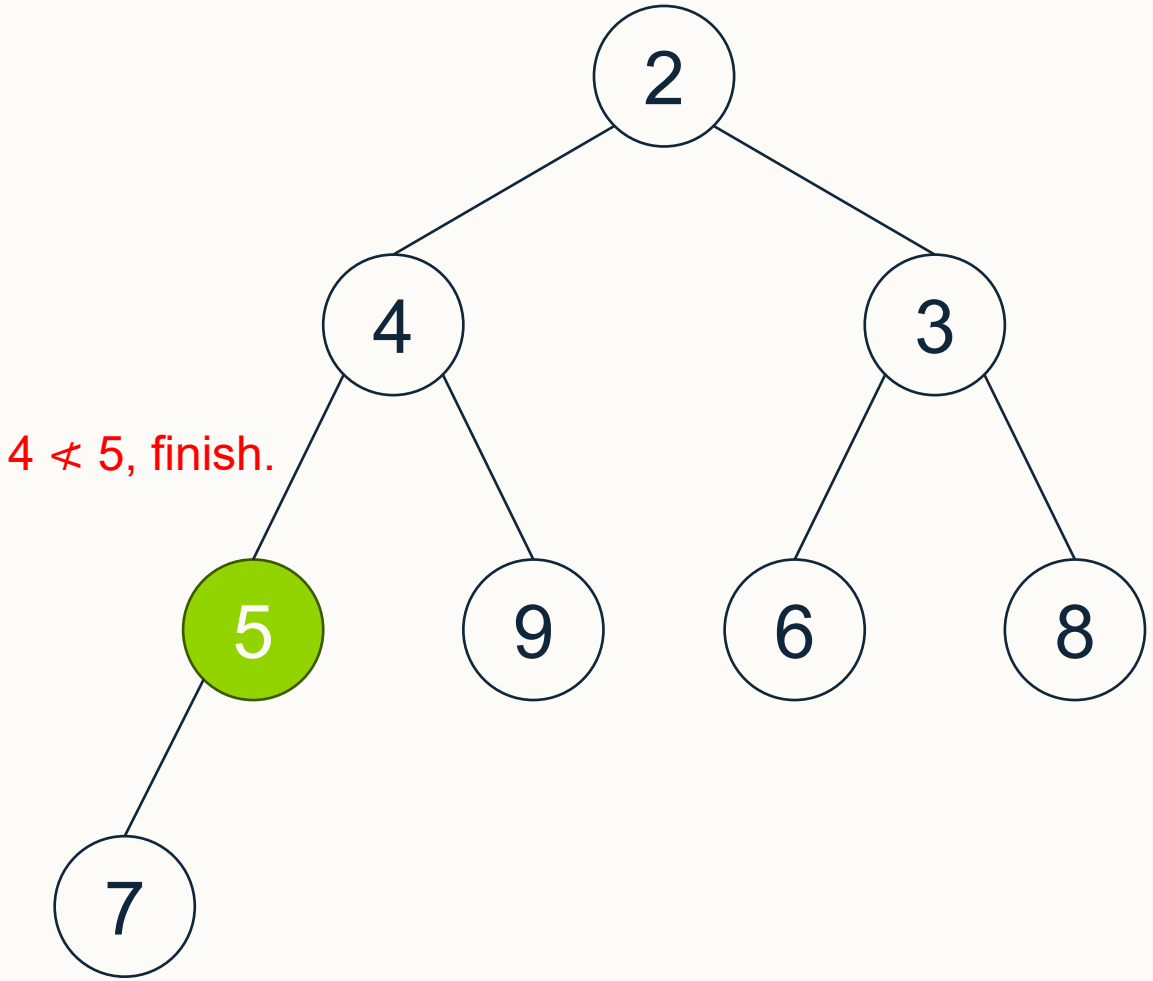
Perform insertItem(5), insertItem(1), then insertItem(10).

Operations on Heaps – insertItem(x)



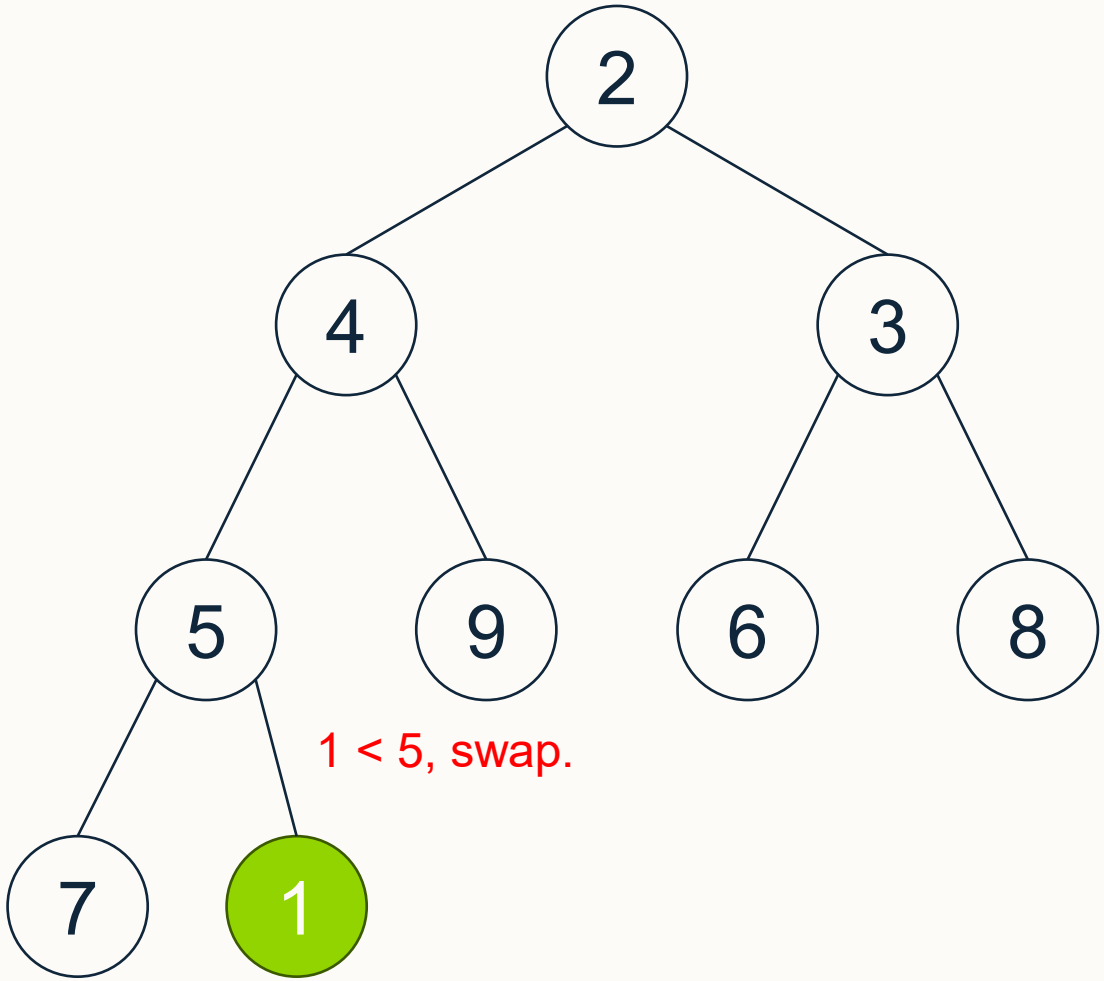
- **InsertItem(5):** The 5 is placed at the left-most available space (left child of 7), and then an up-heap is performed to preserve the heap property.

Operations on Heaps – insertItem(x)



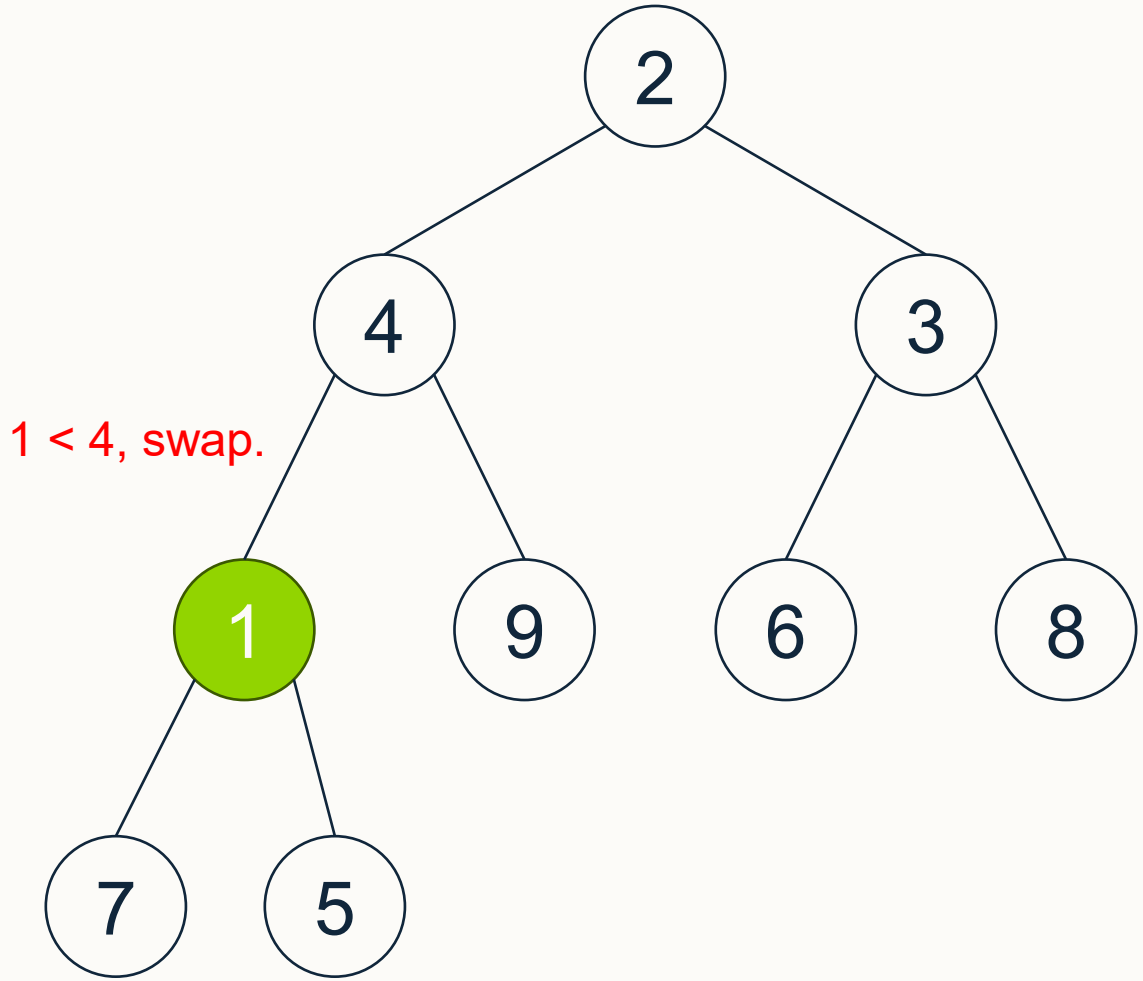
- **InsertItem(5):** The 5 is placed at the left-most available space (left child of 7), and then an up-heap is performed to preserve the heap property.

Operations on Heaps – insertItem(x)



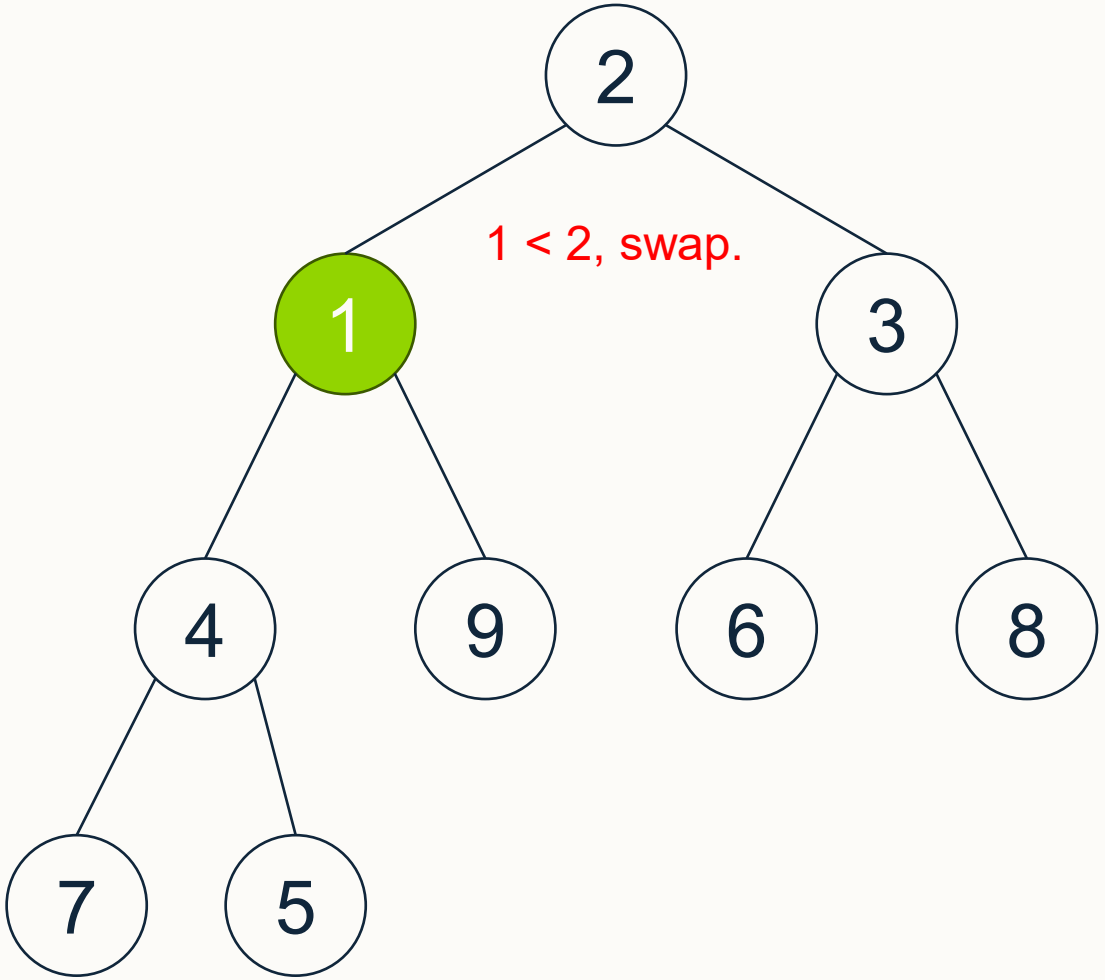
- **InsertItem(1):** The 1 is placed at the left-most available space (right child of 5), and then an up-heap is performed to preserve the heap property.

Operations on Heaps – insertItem(x)



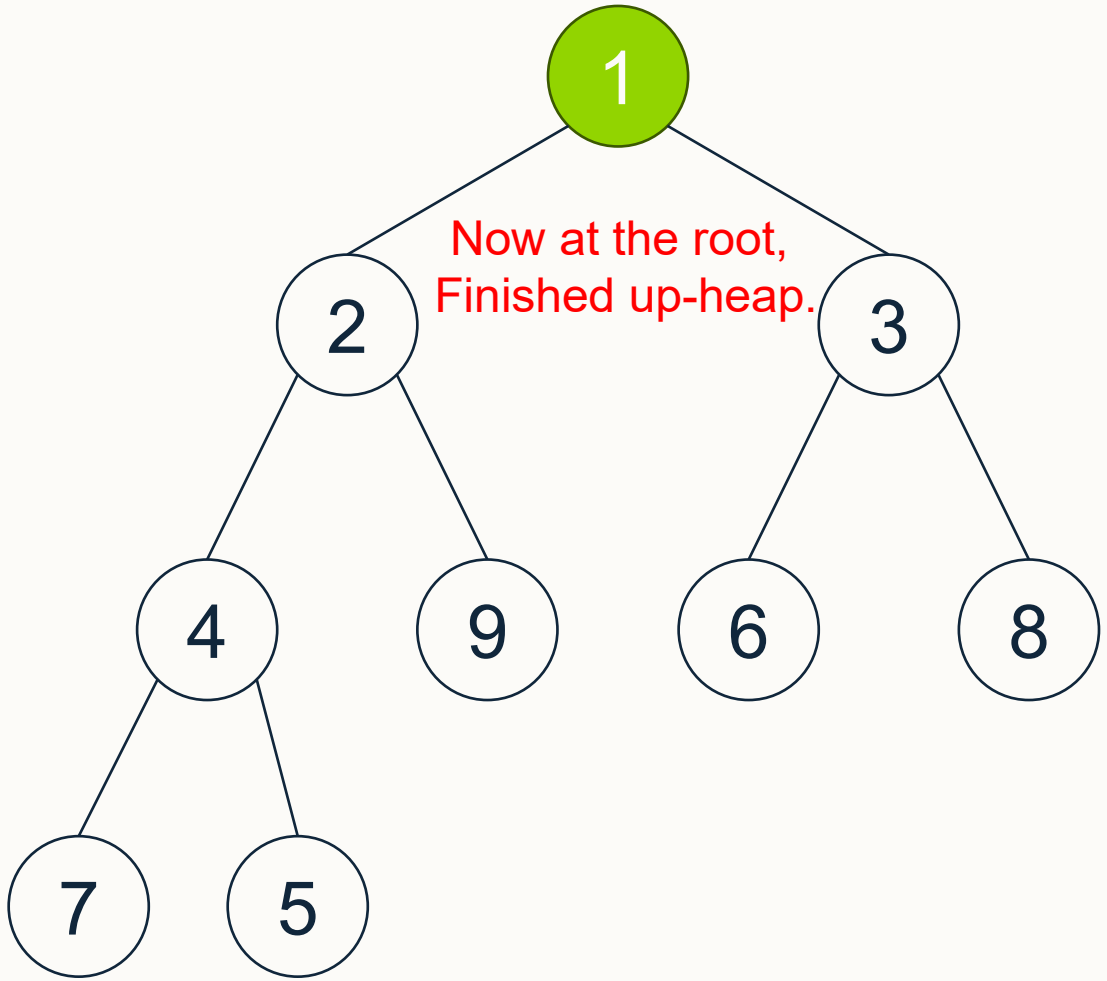
- **InsertItem(1):** The 1 is placed at the left-most available space (right child of 5), and then an up-heap is performed to preserve the heap property.

Operations on Heaps – insertItem(x)



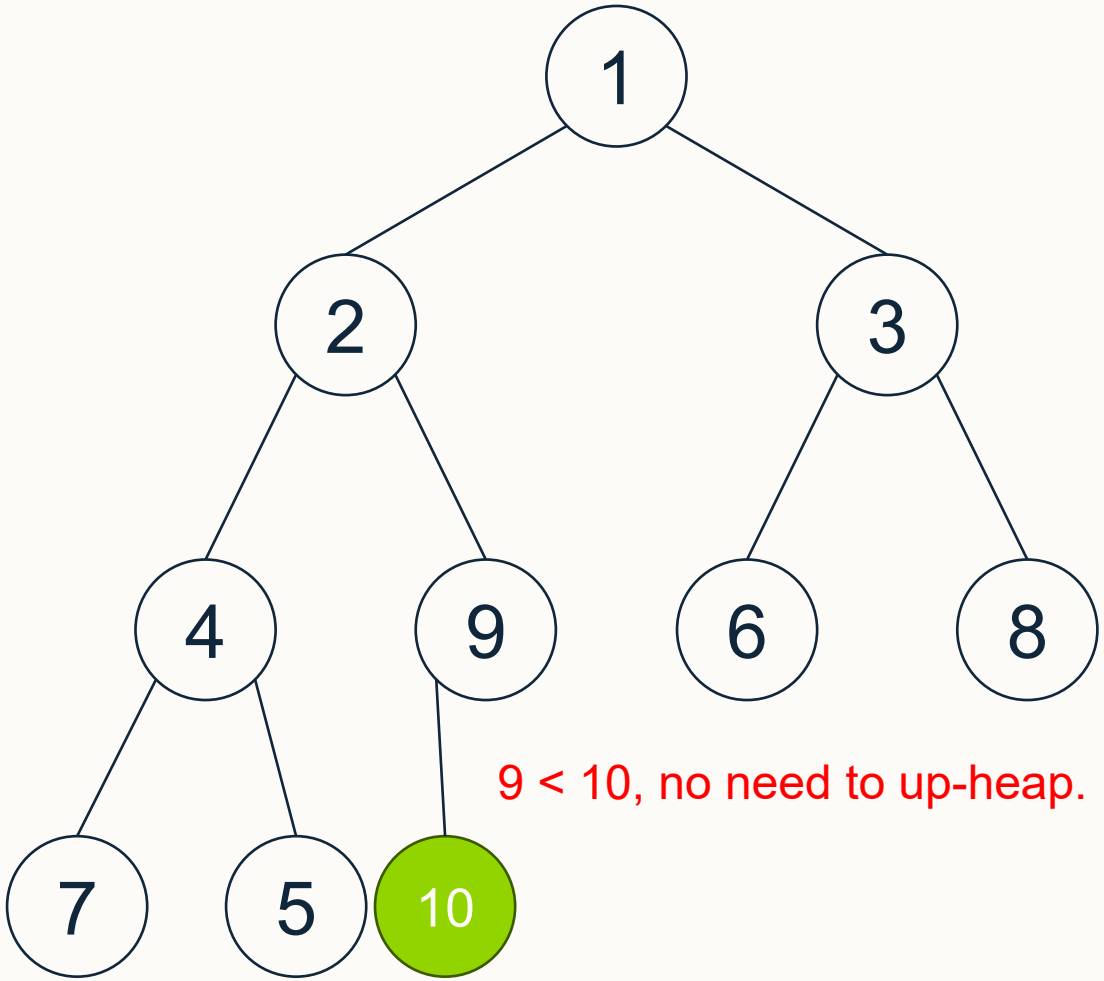
- **InsertItem(1):** The 1 is placed at the left-most available space (right child of 5), and then an up-heap is performed to preserve the heap property.

Operations on Heaps – insertItem(x)



- **InsertItem(1):** The 1 is placed at the left-most available space (right child of 5), and then an up-heap is performed to preserve the heap property.

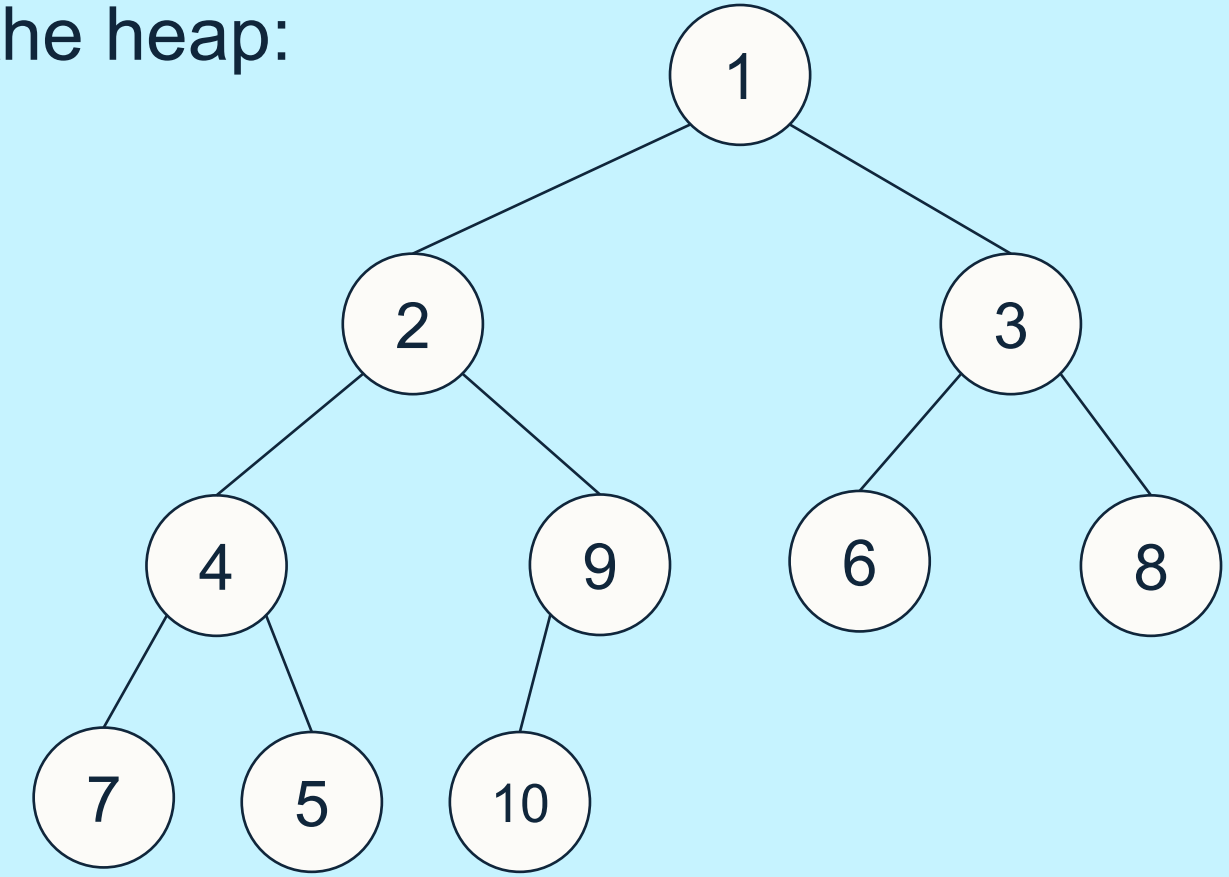
Operations on Heaps – insertItem(x)



- **InsertItem(10):** The 10 is placed at the left-most available space (left child of 9). In this case, there is no need for up-heap.

Operations on Heaps – removeMin()

- Starting with the heap:



Perform removeMin(), swapping with smallest child (if less than current).

Operations on Heaps – removeMin()

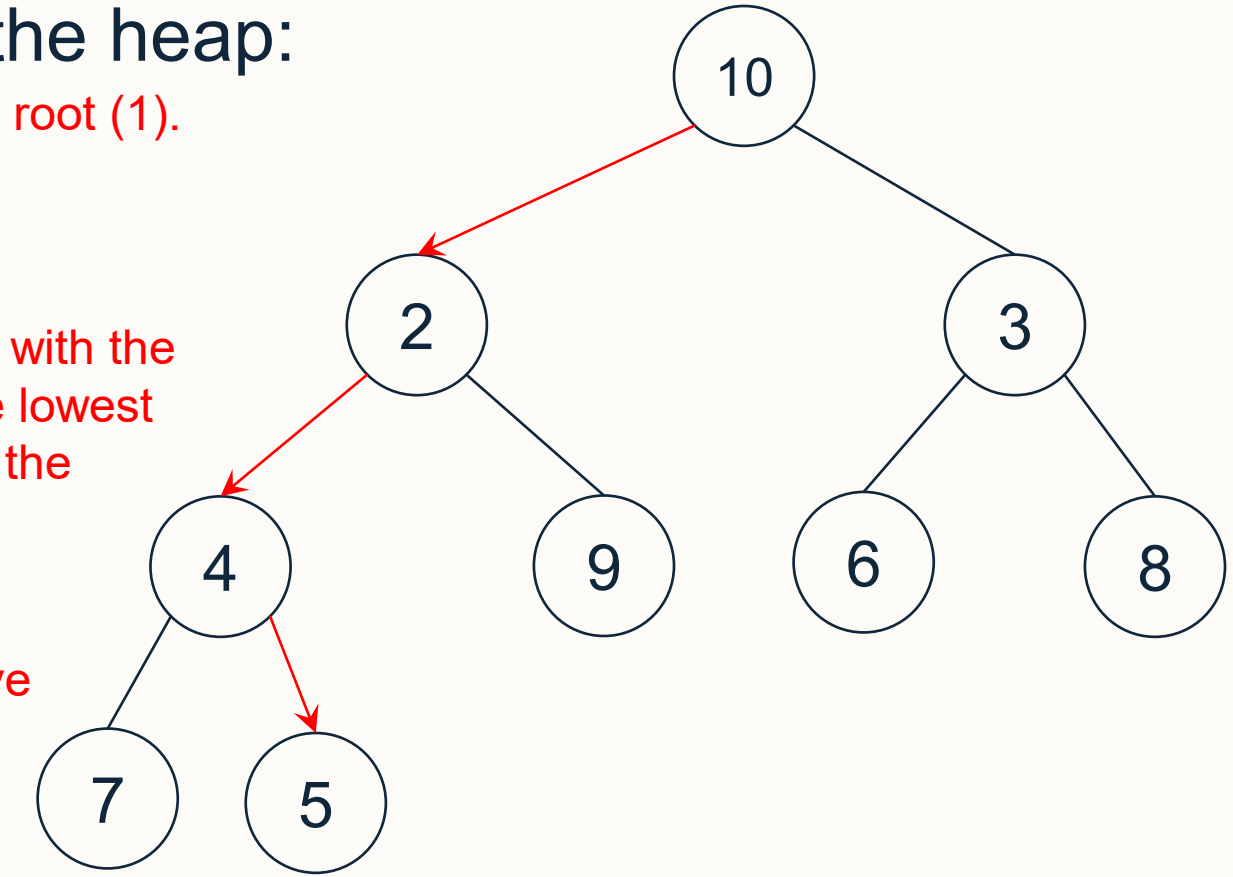
- Starting with the heap:

Start by removing the root (1).

1

Then replace the root with the right-most node in the lowest level (10) to preserve the shape property.

Now need to perform down-heap to preserve the numeric property, swapping with the smallest child.



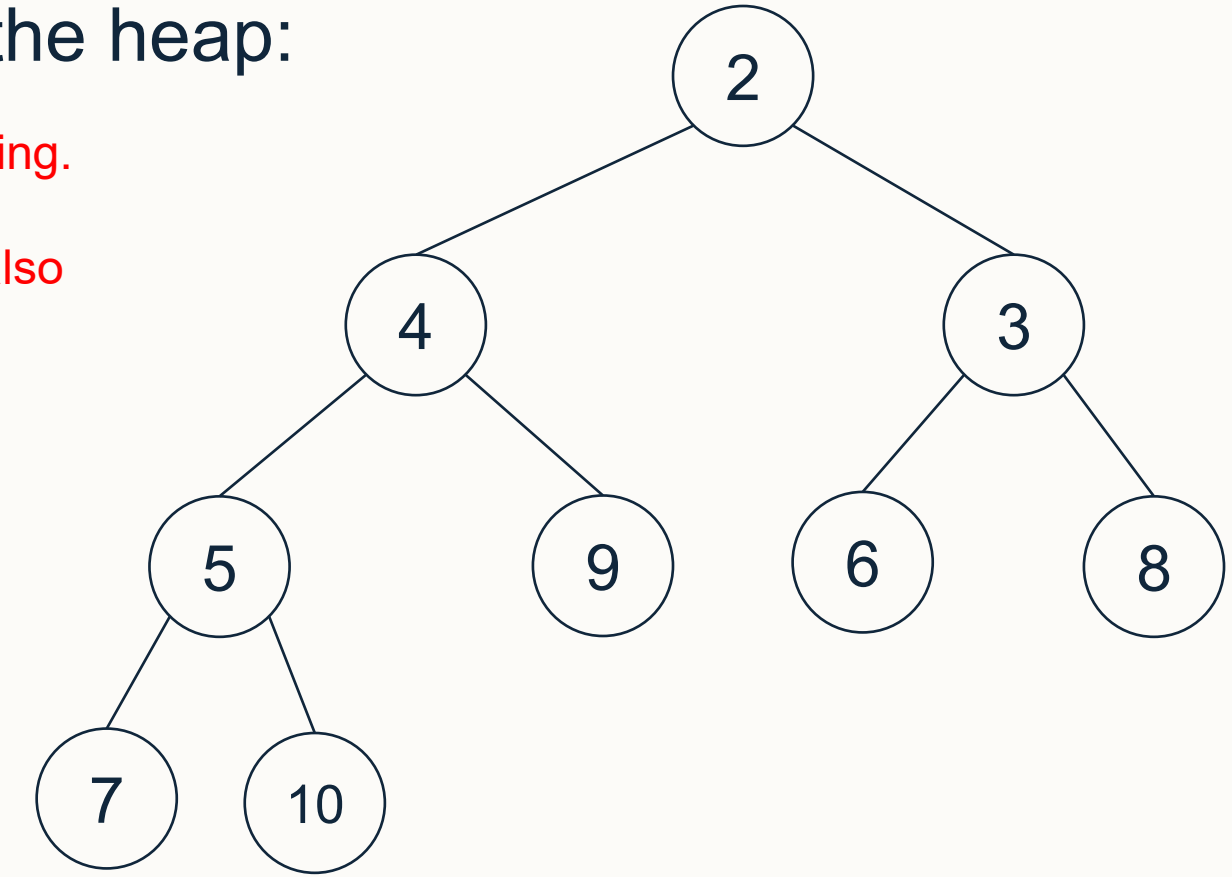
Perform removeMin(), swapping with smallest child (if less than current).

Operations on Heaps – removeMin()

- Starting with the heap:

Which gives the following.

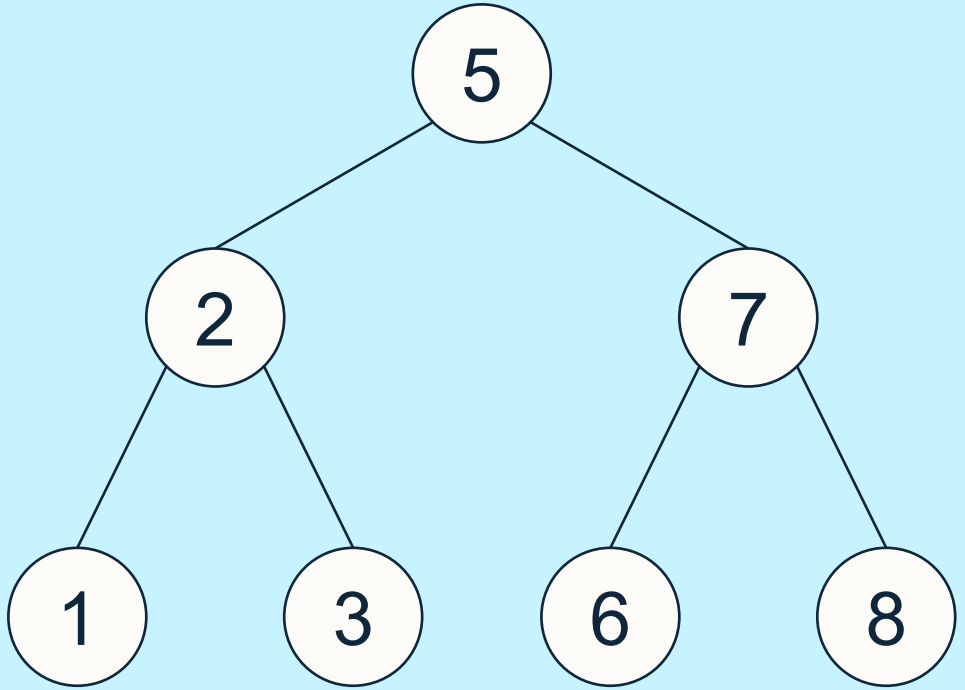
Note how this is now also a valid heap.



Perform removeMin(), swapping with smallest child (if less than current).

Operations on BSTs – insert(x)

- Starting with the BST:



Perform insert(4).

Operations on BSTs – insert(x)

- Starting with the BST:

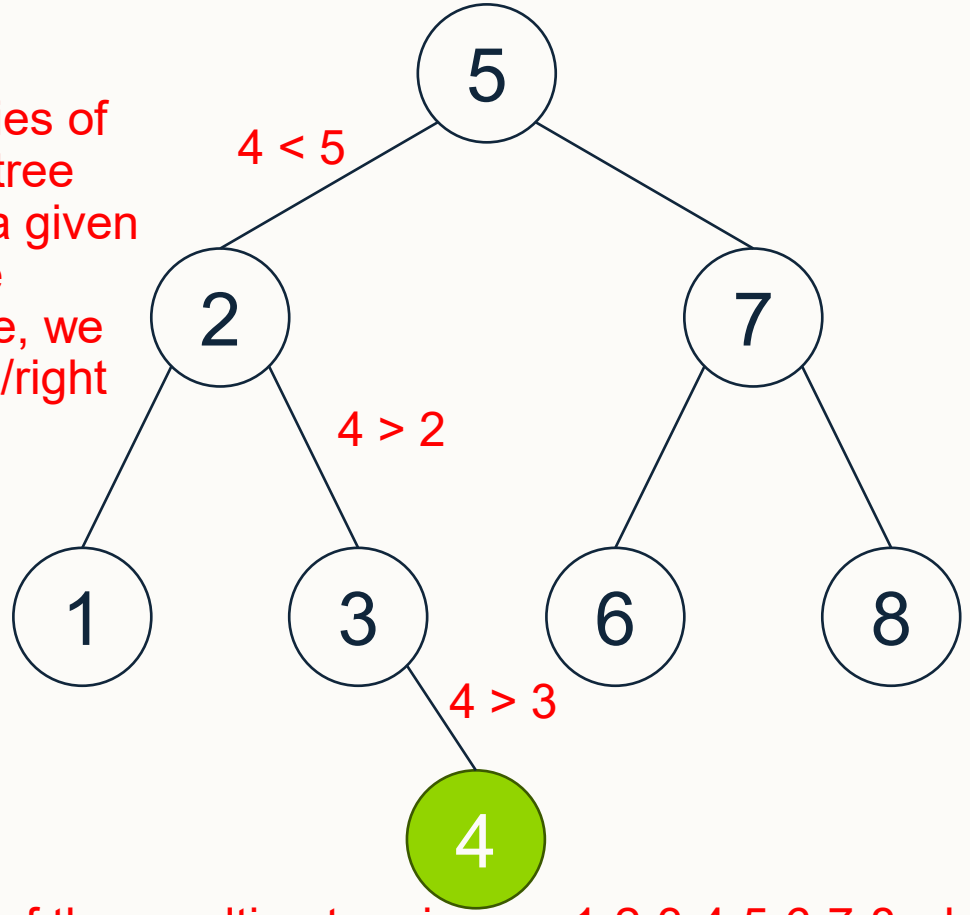
Inserting into a BST is not the same as inserting into a heap.

Since we know that the properties of a BST ensures that the left subtree contains only values less than a given root node, and the right subtree greater than the given root node, we traverse from the root going left/right depending on the value being inserted.

4 < 5 so need to go left.

4 > 2 so need to go right.

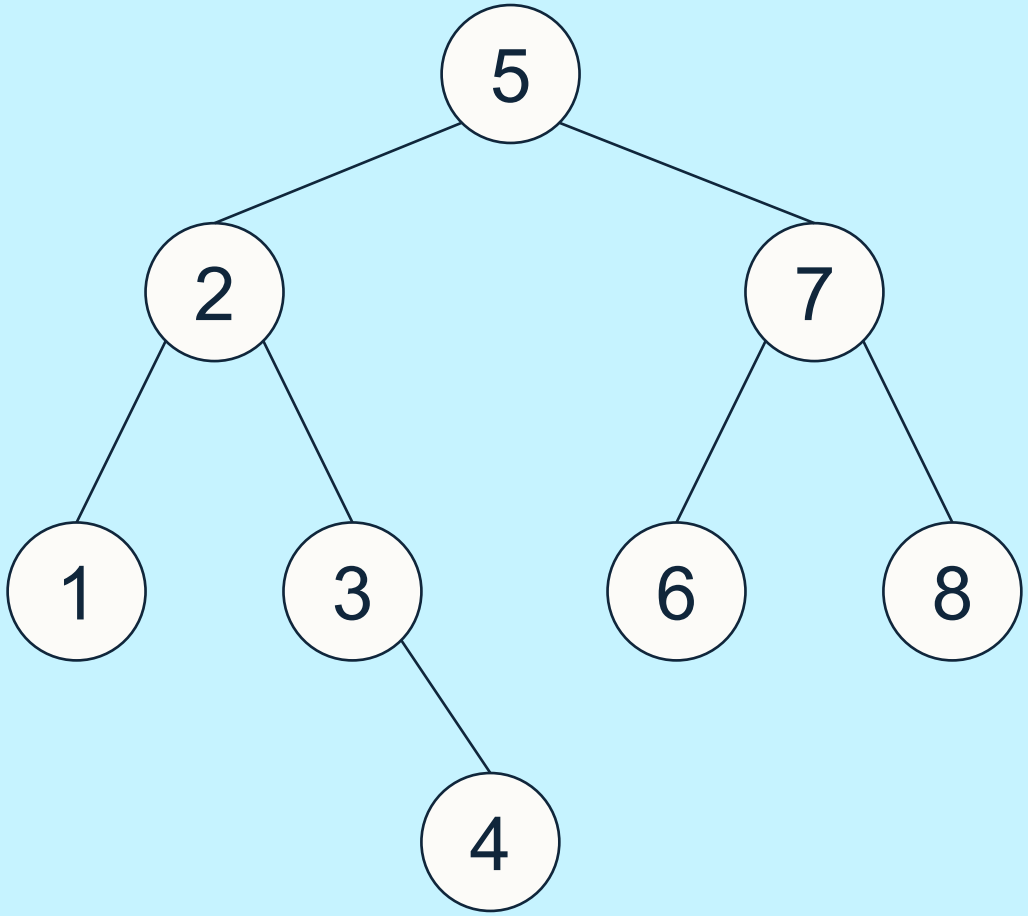
4 > 3 and 3 is a leaf node so make 4 the right child of 3.



Note that the in-order traversal of the resulting tree is now 1-2-3-4-5-6-7-8 which preserves numerical ordering.

Operations on BSTs – remove(x)

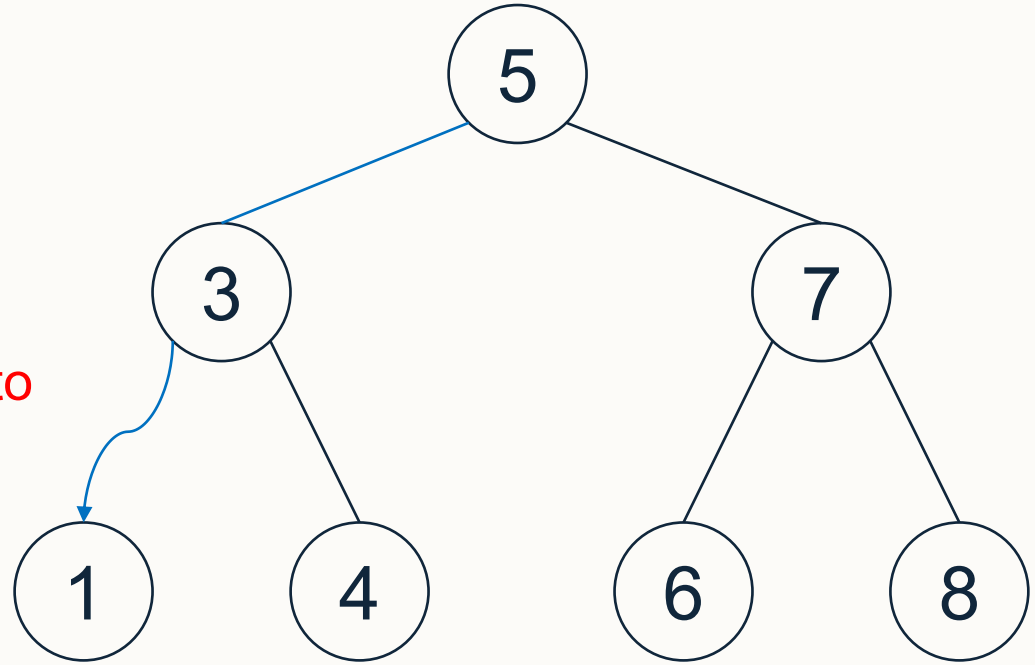
- Starting with the BST:



Perform remove(2), remove(7), then remove(5).

Operations on BSTs – remove(x)

- Starting with the BST:



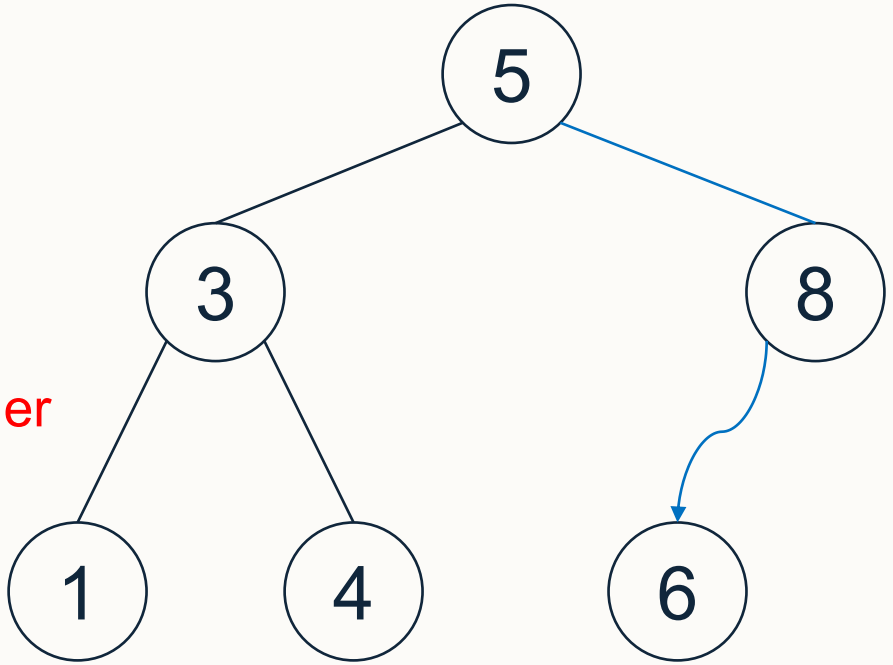
In this example, all of the nodes being removed have 2 children. The 1 child case is trivial – refer to the lecture notes.

Remove(2):

To remove (2), we replace it with the node that follows in the in-order traversal which is (3).

Operations on BSTs – remove(x)

- Starting with the BST:

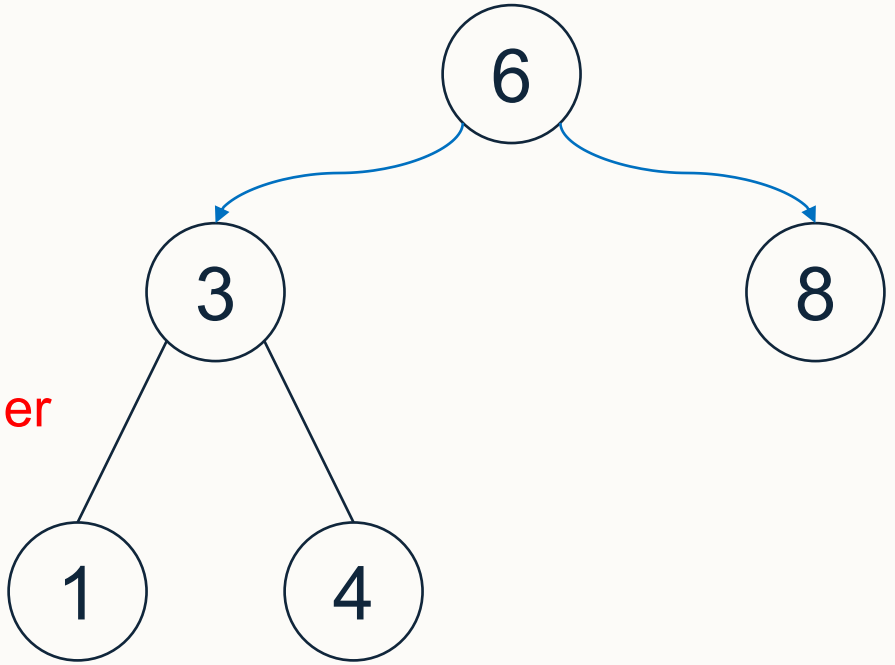


Remove(7):

To remove (7), we replace it with the node that follows in the in-order traversal which is (8).

Operations on BSTs – remove(x)

- Starting with the BST:



Remove(5):

To remove (5), we replace it with the node that follows in the in-order traversal which is (6).

Questions: Heaps and BSTs

Question 1:

a. Draw the following as binary trees and label each with “Heap”, “BST”, or “Neither”: $T_1 = [-, 1,2,4,7,8,5,6]$ and $T_2 = [-, 1,2,5,7,8,4,6]$.
b. If you identify either of the above as a **heap**, then perform `x = removeMin()` followed by `insertItem(x)`. Is the resulting heap identical?

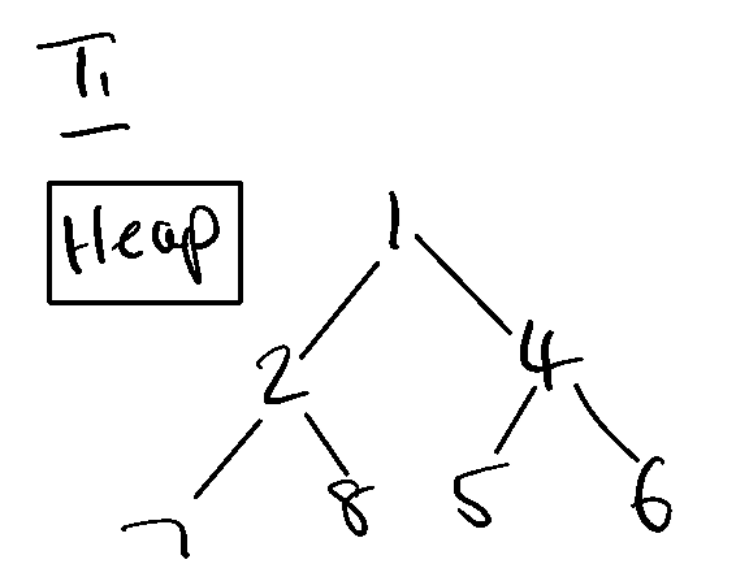
Question 2: Use heapsort to sort the values [2,3,1,4,3’,2’,5]. Using your resulting sorted list, explain if heapsort is stable or not.

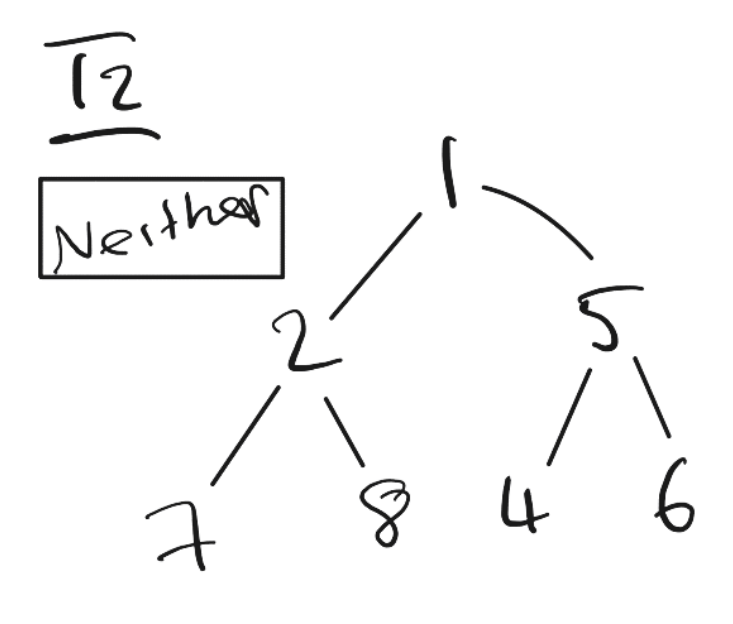
Question 3: With the following BST $[-, 6,2,10,1,5,7,11, -, -, 3, -, -, 9, -, -]$ perform at least the following operations and state the array-based representation after each step:

- Remove(7); remove(2); insert(7); remove(11); remove(10).

Answers: Question 1a

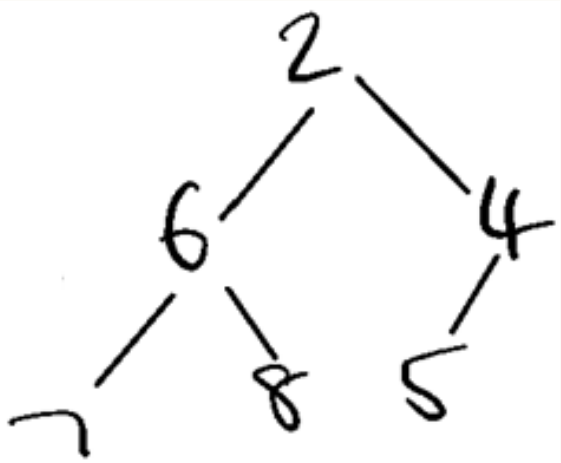
- Draw the following as binary trees and label each with “Heap”, “BST”, or “Neither”: $T_1 = [-, 1,2,4,7,8,5,6]$ and $T_2 = [-, 1,2,5,7,8,4,6]$.



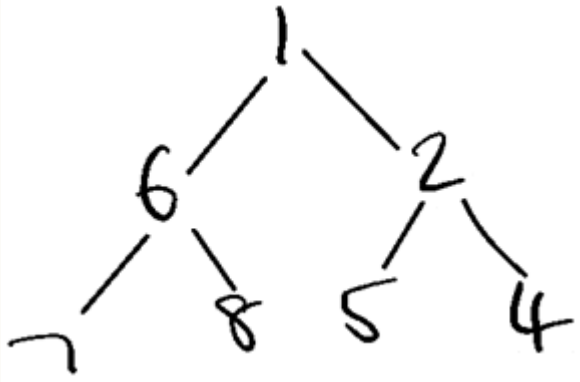


Answers: Question 1b

- If you identify either of these (T_1 and T_2) as a **heap**, then perform `x = removeMin()` followed by `insertItem(x)`.
- Is the resulting heap identical?
- Only T_1 is a heap hence need to perform removeMin() and insertItem(1) which gives the following heap which is not identical.



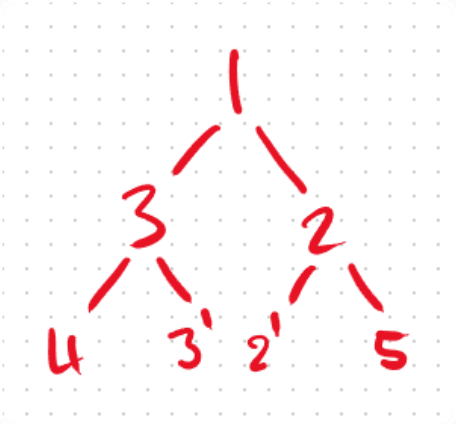
T_1 after removeMin()



T_1 after removeMin() followed by insertItem(1)

Answers: Question 2

- Use heapsort to sort the values [2,3,1,4,3’,2’,5]. Using your resulting sorted list, explain if heapsort is stable or not.
- Adding each element one-by-one into a heap will result in the following heap:

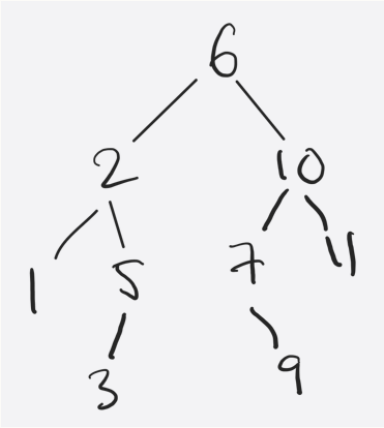


- …and then performing removeMin() 7 times will give the sorted ordering [1, 2, 2’, 3’, 3, 4, 5]. 3’ now appears before 3, which gives a counterexample for heapsort being stable.

Answers: Question 3

With the following BST $[-, 6,2,10,1,5,7,11, -, -, 3, -, -, 9, -, -]$ perform at least the following operations and state the array-based representation after each step:

- Remove(7); remove(2); insert(7); remove(11); remove(10).
- Drawn as a tree, the BST is:



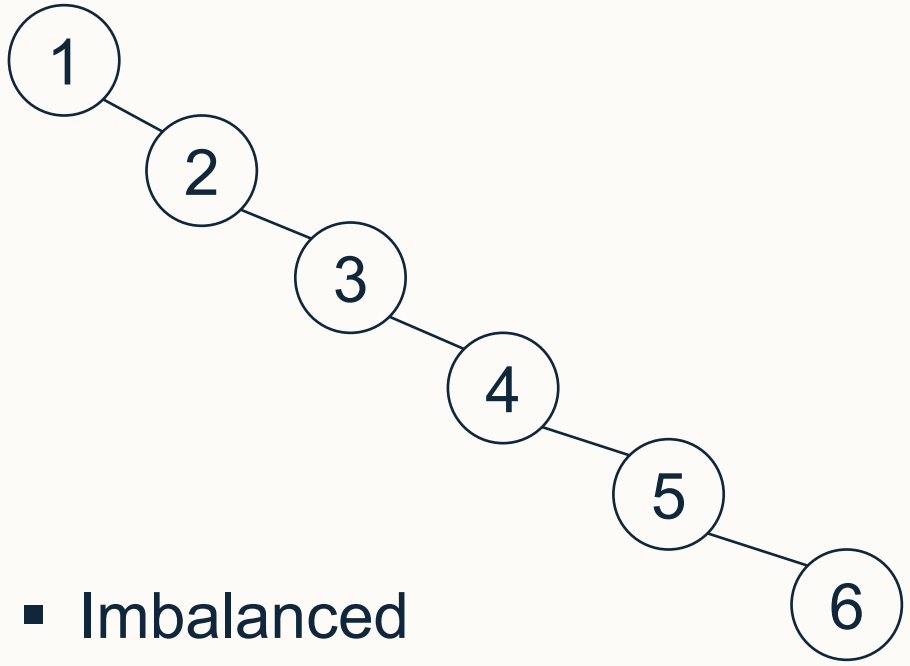
- Remove(7): the 7 is replaced with its single child 9 to give $[-, 6,2,10,1,5,9,11, -, -, 3, -, -, -, -, -]$.
- Remove(2): the 2 is replaced with the node 3 that follows in an in-order traversal to give $[-, 6,3,10,1,5,9,11, -, -, -, -, -, -, -, -]$.
- Insert(7): 7 > 6 so go to the right sub-tree; 7 < 10 so go to the left sub-tree, 7 < 9 and 9 has no left child so add 7 as the left child of 9 to give $[-, 6,3,10,1,5,9,11, -, -, -, -, 7, -, -, -]$.
- Remove(11): 11 has no children so can remove it to give $[-, 6,3,10,1,5,9, -, -, -, -, -, 7, -, -, -]$.
- Remove(10): 10 has one left child which itself has a left child. Set the parent of 9 to be 6 and the right child of 6 to be 9 to give $[-, 6,3,9,1,5,7, -, -, -, -, -, -, -, -, -]$.

University of Nottingham
UK | CHINA | MALAYSIA

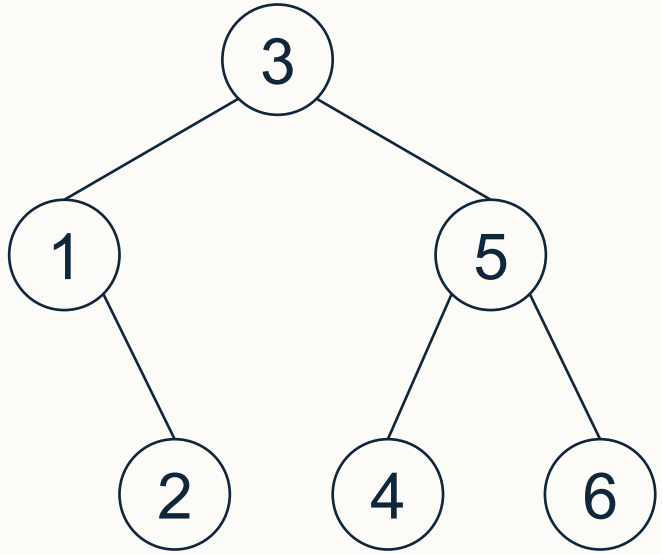
Rotations with BSTs

Motivation

- Important BST algorithms are O(height)
- For example, for a tree with $n = 6$ nodes:



- Imbalanced
- height = 5
- Operations will be O(n)



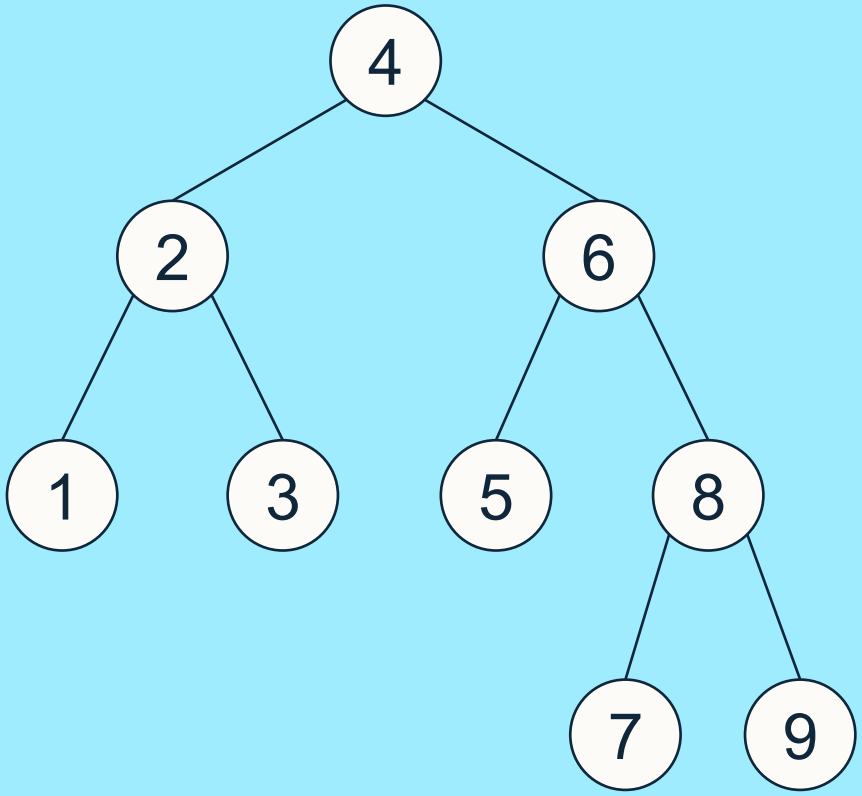
- Balanced
- height = 2
- Operations will be O(log n)

BST Rotations

- **Aim:** rebuild part of the tree in O(log n) time, so that the balance is maintained following insertion/deletion operations
- Resulting tree must have the same in-order traversal

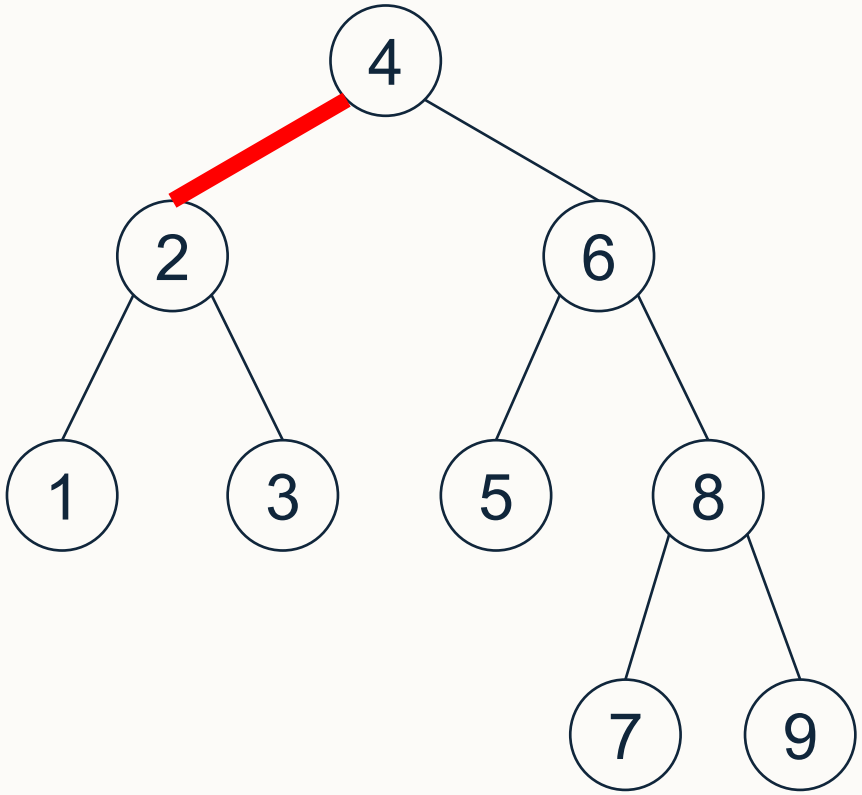
Single Rotation Example 1

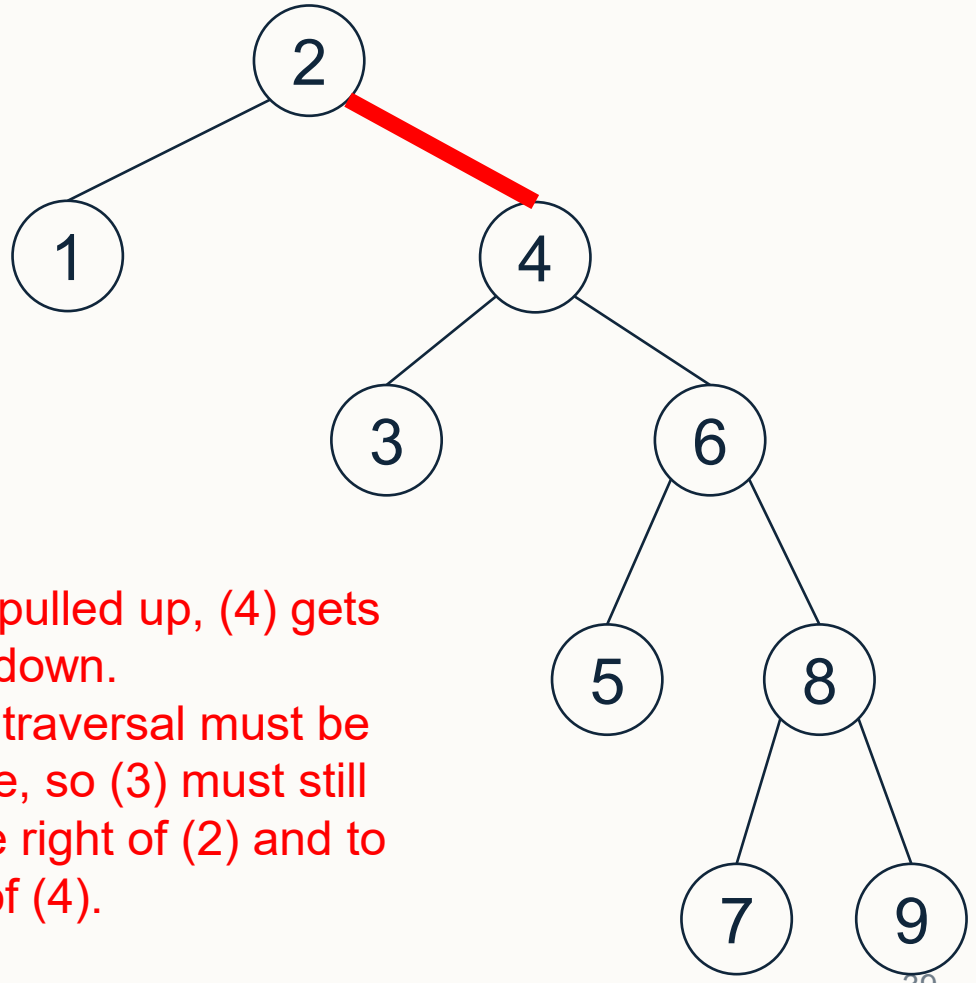
Rotate with edge 2-4



Single Rotation Example 1

Rotate with edge 2-4

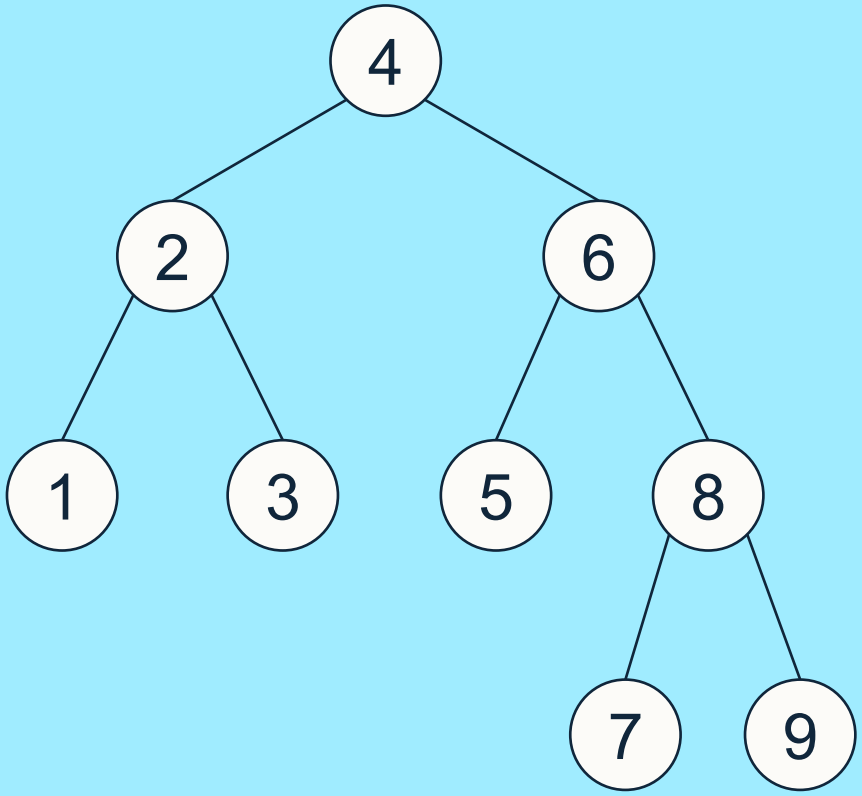




- (2) gets pulled up, (4) gets pushed down.
- In-order traversal must be the same, so (3) must still be to the right of (2) and to the left of (4).

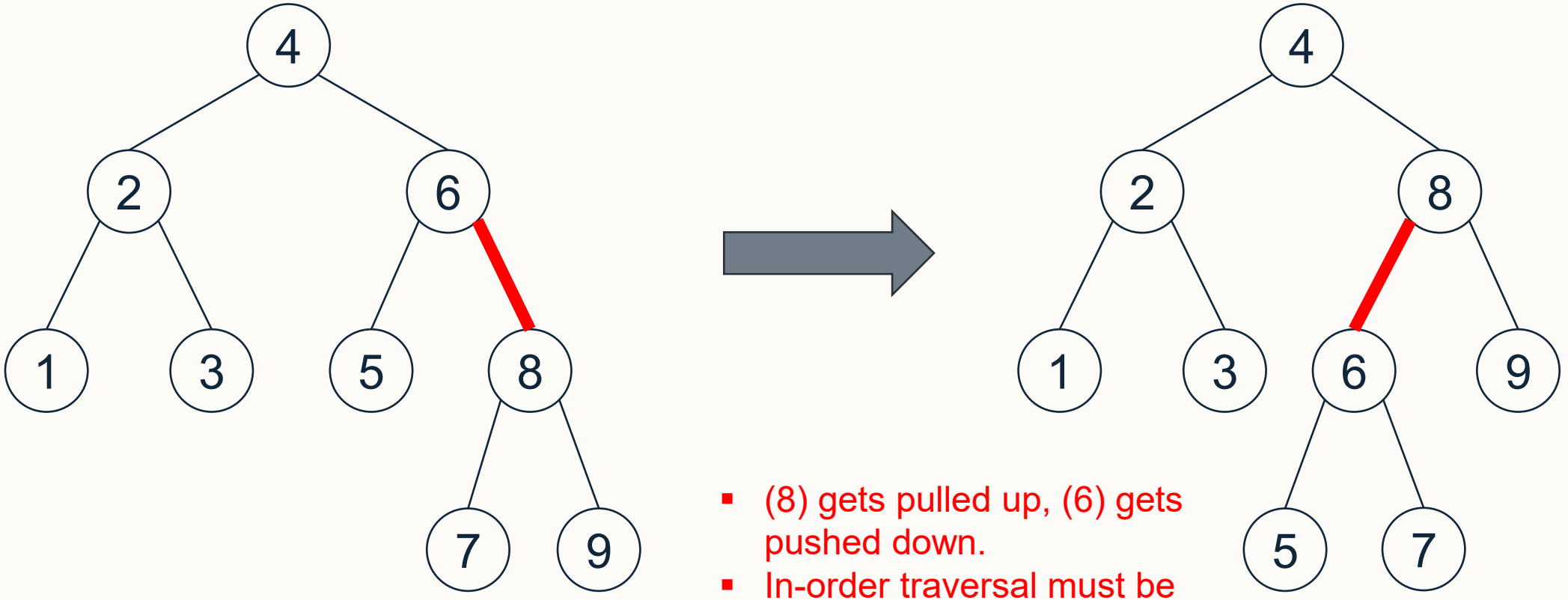
Single Rotation Example 2

Rotate with edge 6-8



Single Rotation Example 2

Rotate with edge 6-8



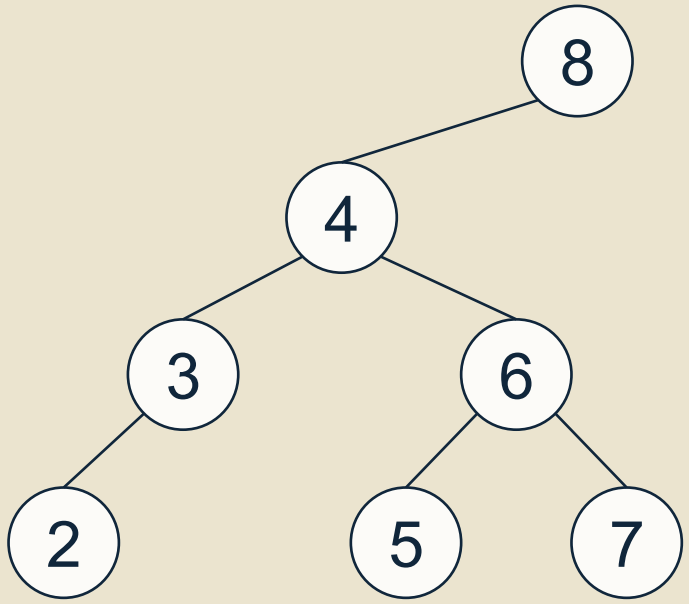
- (8) gets pulled up, (6) gets pushed down.
- In-order traversal must be the same, so (7) must still be to the right of (6) and to the left of (8).

Questions

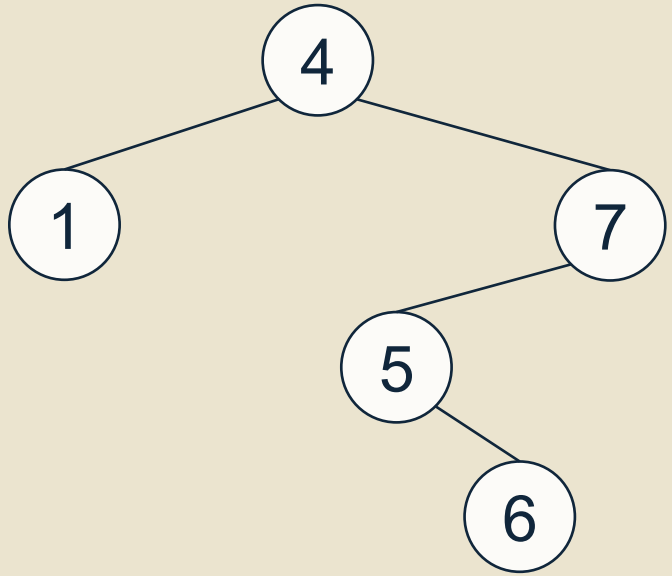
Q1. Starting from the BST to the right:

a. Insert 1 (see Tutorial 6 if you need a reminder of BST insertions!)

b. Perform a **single rotation** to reduce the height of the resulting tree



Q2. Given the BST to the right, find the **two pairs** of single rotations that **minimise** the height of the tree. Perform both pairs of rotations.

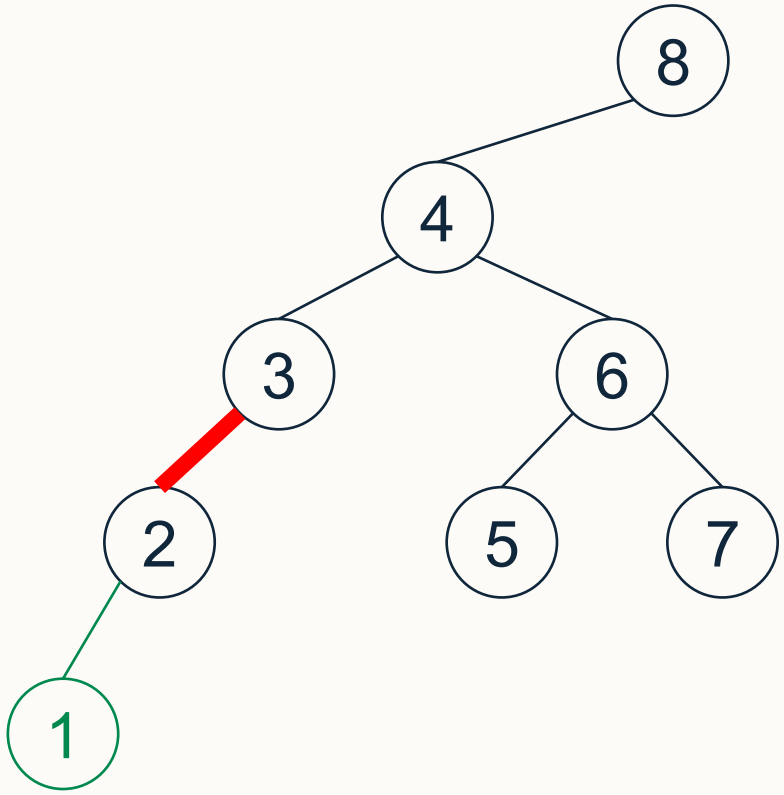


Question 1 - Answer

Q1. Starting from the BST to the right:

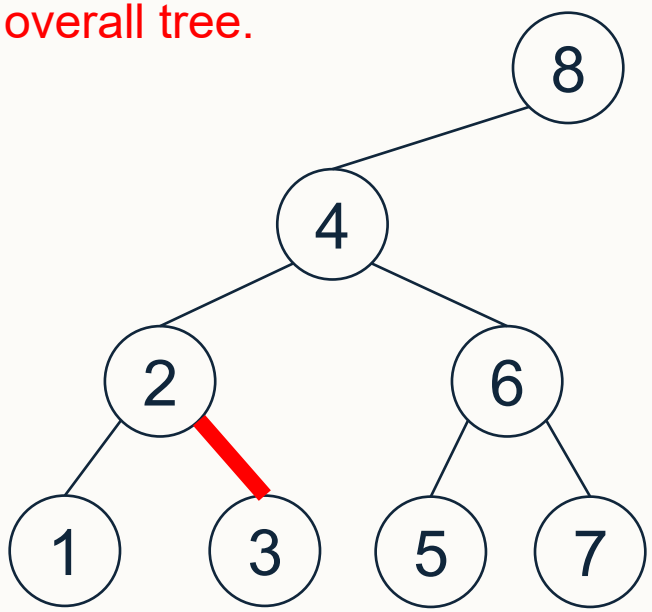
a. Insert 1 (see Tutorial 6 if you need a reminder of BST insertions!)

b. Perform a **single rotation** to reduce the height of the resulting tree



The subtree containing (1), (2) and (3) is imbalanced, and has median value (2). Making (2) the root of this subtree will balance the subtree, and reduce the height of the overall tree.

Rotate with edge 2-3:

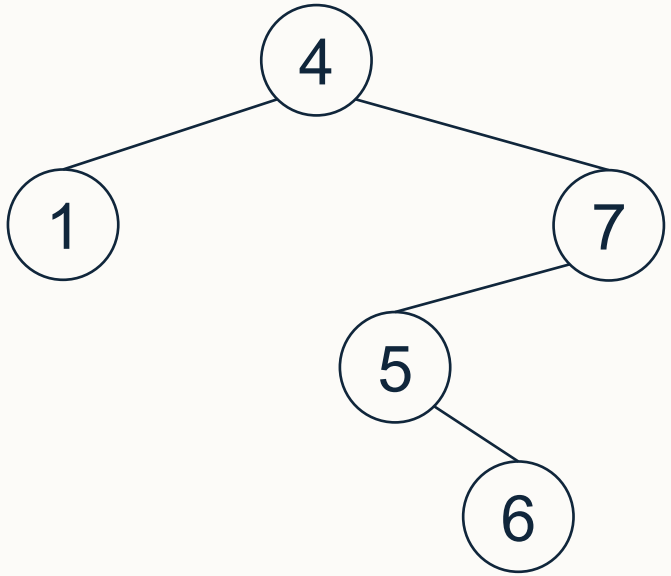


Note: rotating with edge 4-8 would also be a valid choice to reduce the height!

Question 2 - Answer

Q2. Given the BST to the right, find the **two pairs** of single rotations that **minimise** the height of the tree. Perform both pairs of rotations.

Pair 1: considering the whole tree, 5 is the median value so ideally would be at the root. Each single rotation can move 5 up by at most one level in the tree, so by doing two single rotations we can move 5 to the root.
Rotate edge 5-7, rotate edge 4-5



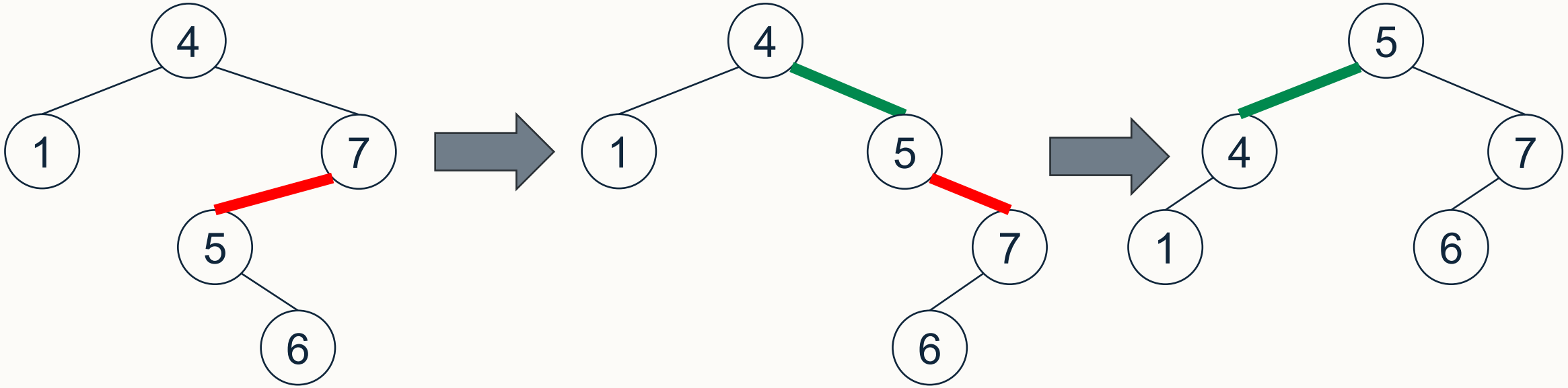
Pair 2: considering the subtree containing (5), (6), (7), the median value is 6. Ideally 6 would be at the root of this subtree. Again, we can achieve this by doing two single rotations.
Rotate edge 5-6, rotate edge 6-7

Question 2 - Answer

Q2. Given the BST to the right, find the **two pairs** of single rotations that **minimise** the height of the tree. Perform both pairs of rotations.

Pair 1:

- Rotate edge 5-7
- Rotate edge 4-5

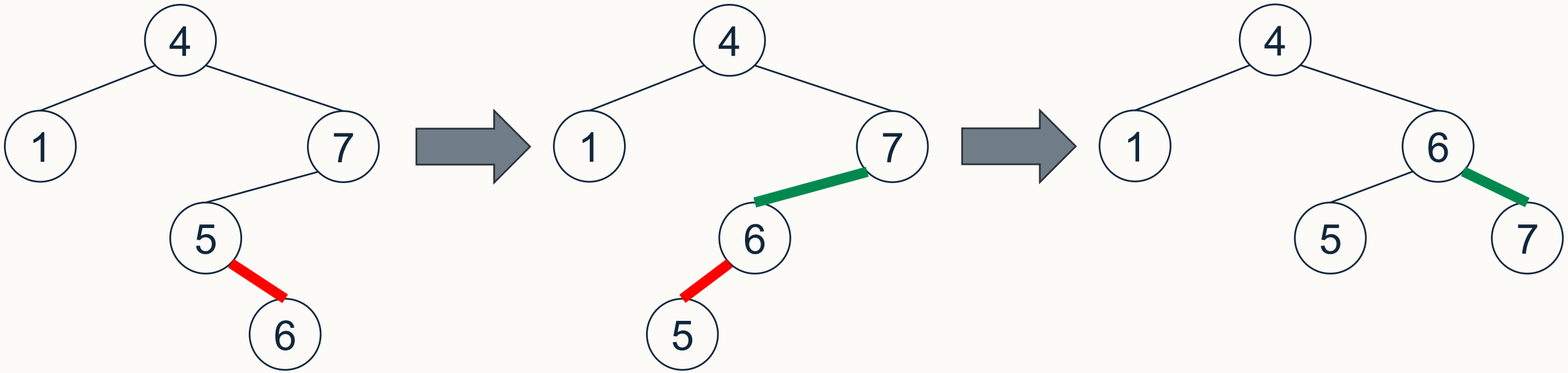


Question 2 - Answer

Q2. Given the BST to the right, find the **two pairs** of single rotations that **minimise** the height of the tree. Perform both pairs of rotations.

Pair 2:

- Rotate edge 5-6
- Rotate edge 6-7



University of Nottingham
UK | CHINA | MALAYSIA

Thank you